

UNIVERSITÉ DE LORRAINE

Rapport de Preuve et Dédution Automatique

Implémentation de la méthode des connexions en Prolog

Étudiants :

Mathéo VIGNÈRES

Guillaume POTEL

Enseignant :

M. Didier GALMICHE

Année universitaire 2025-2026

Table des matières

Introduction	2
1 Le socle commun : Logique Propositionnelle et Premier Ordre	2
1.1 Définition des règles (Alpha et Bêta)	2
1.1.1 Le prédicat principal : <code>etiqueter/7</code>	2
1.1.2 Les règles de type Alpha (α)	2
1.1.3 Les règles de type Bêta (β)	2
1.1.4 Cas de la négation et des feuilles	3
1.2 Structures de données : Représentation des arbres	3
1.2.1 Une structure récursive uniforme	3
1.2.2 Polyvalence de la structure	4
1.3 Prédicats utilitaires	4
1.3.1 Identification des littéraux et opérateurs	4
1.3.2 Substitution de termes	4
1.3.3 Gestion de l’affichage et de la trace	5
2 En Logique Propositionnelle	5
2.1 Construction de l’arbre syntaxique indexé	5
2.1.1 Structure de l’étiquette enrichie	5
2.1.2 Initialisation et Génération	5
2.1.3 Récupération par les accesseurs	6
2.2 Génération de l’arbre des chemins	6
2.2.1 Le développement de chemin	6
2.2.2 Traitement des règles α et β	6
2.2.3 Sémantique de l’Arbre des Chemins	7
2.3 Recherche de connexions et validation	7
2.3.1 Définition d’une connexion	7
2.3.2 Validation globale des chemins	7
2.3.3 Conclusion de la preuve	8
3 En Logique du Premier Ordre (LPO)	8
3.1 Adaptation de l’arbre indexé	8
3.2 Évolution de l’arbre des chemins	8
3.3 Recherche de connexions orientée LPO	8
3.4 Unification, graphe de dépendance et multiplicité	8
Conclusion	8

Introduction

1 Le socle commun : Logique Propositionnelle et Premier Ordre

Dans cette première partie, nous présentons les éléments de base qui sont partagés entre la logique propositionnelle et la logique du premier ordre.

1.1 Définition des règles (Alpha et Bêta)

La méthode des connexions s'appuie sur l'application de règles appelée *alpha* et *beta*. Nous avons implémenté ces règles via un prédicat central : `etiqueter/7`.

Ce prédicat a pour rôle de parcourir la formule initiale pour construire un arbre indexé, où chaque nœud contient le type de règle appliquée ($\alpha, \beta, \gamma, \delta$), sa polarité, et un index unique.

1.1.1 Le prédicat principal : `etiqueter/7`

La signature du prédicat est la suivante :

```
1 etiqueter(+Formule, +Polarite, +IndexIn, -IndexOut, +Multiplicite,  
+Suffixe, -Arbre).
```

- **Polarité** : 1 pour une formule affirmée, 0 pour une formule niée.
- **IndexIn / IndexOut** : Un mécanisme de compteur permettant d'attribuer un identifiant unique (ex : $a0, a1, \dots$) à chaque nœud de l'arbre.
- **Arbre** : La structure de sortie représentant l'arbre syntaxique indexé sous forme de nœud ou de feuille.

1.1.2 Les règles de type Alpha (α)

Les règles α correspondent aux décompositions de nature conjonctive (linéaire). Dans notre implémentation, une règle α produit deux sous-arbres qui devront tous deux être validés. Par exemple, pour la conjonction de polarité 1 ($A \wedge B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 0$) :

```
1 % Exemple : Implication niee (A -> B, 0)  
2 etiqueter(A impl B, 0, IndexIn, IndexOut, M, Suffix,  
3     noeud(etiq_formule(alpha, alpha1, alpha2, A impl B, 0,  
4         NomComplet),  
5         [ArbreA, ArbreB])) :-  
6     construire_nom(IndexIn, Suffix, NomComplet),  
7     Index1 is IndexIn + 1,  
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),  
     etiqueter(B, 0, IndexMid, IndexOut, M, Suffix, ArbreB).
```

1.1.3 Les règles de type Bêta (β)

Les règles β représentent des décompositions disjonctives (branchement). Elles créent une bifurcation dans l'arbre des chemins. Nous retrouvons ici la disjonction de polarité 1 ($A \vee B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 1$) :

```

1 % Exemple : Disjonction affirmée (A ou B, 1)
2 etiqueter(A ou B, 1, IndexIn, IndexOut, M, Suffix,
3     noeud(etiq_formule(beta, beta1, beta2, A ou B, 1,
4         NomComplet),
5         [ArbreA, ArbreB])) :-
6     construire_nom(IndexIn, Suffix, NomComplet),
7     Index1 is IndexIn + 1,
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),
9     etiqueter(B, 1, IndexMid, IndexOut, M, Suffix, ArbreB).

```

1.1.4 Cas de la négation et des feuilles

La négation est traitée de manière transparente en inversant la polarité du reste de la formule. Ce processus se poursuit jusqu'à atteindre un **littéral** (atome). À ce stade, le prédicat crée une feuille, marquant la fin de la branche syntaxique.

```

1 % Traitement des atomes (Feuilles)
2 etiqueter(A, Polarite, Index, IndexOut, _, Suffix,
3     feuille(etiq_formule(atome, none, none, A, Polarite,
4         NomComplet)))) :-
5     est_litteral(A),
6     construire_nom(Index, Suffix, NomComplet),
7     IndexOut is Index + 1.

```

1.2 Structures de données : Représentation des arbres

Pour représenter un arbre dans notre projet, nous avons choisi une structure récursive polyvalente capable de représenter aussi bien l'arbre syntaxique indexé que l'arbre des chemins.

1.2.1 Une structure récursive uniforme

Un arbre est défini de manière inductive par deux constructeurs principaux :

- `noeud(Etiquette, ListeFils)` : Représente un point de divergence ou de linéarité dans la formule. Contrairement à un arbre binaire classique, nous utilisons une liste de fils pour permettre une plus grande flexibilité (notamment pour la gestion de la multiplicité en LPO).
- `feuille(Etiquette)` : Représente un point terminal, (un littéral).

L'étiquette contient les métadonnées nécessaires au traitement : pour l'arbre syntaxique, il s'agit du prédicat `etiq_formule` vu précédemment. Pour l'arbre des chemins, elle contient l'ensemble des littéraux collectés.

```

1 % Verification du type de noeud
2 est_feuille(feuille(_)).
3 est_noeud(noeud(_, _)).
4
5 % Accesseurs pour l'extraction de donnees
6 noeud_etiquette(noeud(E, _), E).
7 noeud_fils(noeud(_, Fils), Fils).
8 feuille_etiquette(feuille(E), E).

```

1.2.2 Polyvalence de la structure

Cette implémentation est utilisée à deux étapes clés du programme :

1. **L'Arbre Indexé** : Chaque nœud correspond à une sous-formule et chaque feuille à un atome avec sa polarité. C'est la base de la décomposition logique.
2. **L'Arbre des Chemins** : Il s'agit d'une transformation de l'arbre précédent où chaque nœud représente l'état des chemins à un point donné de l'exploration.

Cette uniformité de structure facilite l'écriture des algorithmes de parcours de profondeur, car les mêmes prédicats de navigation peuvent être réutilisés sur les deux types d'arbres.

1.3 Prédicats utilitaires

Pour soutenir les étapes de décomposition et de recherche de preuves, nous avons développé un ensemble d'outils. Ces prédicats gèrent la syntaxe logique, la manipulation des variables et les mécanismes d'affichage.

1.3.1 Identification des littéraux et opérateurs

Un point crucial de la méthode des connexions est de savoir quand arrêter la décomposition d'une formule. Le prédicat `est_litteral/1` permet d'identifier si un terme est un atome (propositionnel ou prédicat du premier ordre) en vérifiant qu'il n'est pas un connecteur logique réservé.

```
1 % Un terme est un littéral s'il n'est pas un connecteur logique
2 est_litteral(A) :-
3     atom(A), !.
4 est_litteral(A) :-
5     compound(A),
6     functor(A, Foncteur, _),
7     \+ connecteur(Foncteur).
```

Nous avons également défini les priorités d'opérateurs pour permettre une écriture naturelle des formules (ex : `p et q impl r`).

1.3.2 Substitution de termes

Particulièrement utilisé en logique du premier ordre pour l'application des règles γ (universelles) et δ (existentielles), le prédicat `substituer/4` effectue un remplacement récursif d'un terme par un autre.

```
1 % Remplacement récursif de Ancien par Nouveau dans Terme
2 substituer(Ancien, Ancien, Nouveau, Nouveau) :- !.
3 substituer(Terme, Ancien, Nouveau, Resultat) :-
4     compound(Terme),
5     Terme =.. [F | Args],
6     maplist([Arg, ArgSubst]>>(substituer(Arg, Ancien, Nouveau,
7     ArgSubst)), Args, ArgsSubst),
8     Resultat =.. [F | ArgsSubst].
9 substituer(Terme, _, _, Terme).
```

Ce prédicat utilise l'opérateur "univ" ($=.$) pour décomposer n'importe quel prédicat en une liste de ses arguments, permettant une substitution générique quelle que soit l'arité du terme.

1.3.3 Gestion de l'affichage et de la trace

Nous avons créé tout un module d'affichage pour suivre les traces de la méthode des connexions. Pour ne pas encombrer la logique métier, ces outils sont regroupés et traitent les points suivants :

- **Écriture symbolique** : Les prédicats `ecrire_formule/1` et `ecrire_type/1` traduisent les termes internes (ex : `impl`, `alpha`) en symboles mathématiques ($\rightarrow$, α , $\forall$, $\exists$).
- **Nettoyage des variables** : `nettoyer_formule/2` transforme les structures internes complexes (utilisées pour l'unification) en noms lisibles pour l'utilisateur final.
- **Système d'Echo** : Nous avons réutilisé le système d'affichage donné pour le projet d'algorithme d'unification de Martelli-Montanari au 1er semestre, dans le cadre de l'UE *Logique et Modèles de Calculs*.

2 En Logique Propositionnelle

Cette section détaille l'implémentation de la méthode des connexions restreinte à la logique propositionnelle.

2.1 Construction de l'arbre syntaxique indexé

La première étape de la méthode consiste à transformer la formule logique brute en un arbre syntaxique dont chaque nœud contient les informations nécessaires à la suite de la méthode. Cette structure, appelée **Arbre Indexé**, sert de base à toutes les étapes ultérieures (génération des chemins et recherche de connexions).

2.1.1 Structure de l'étiquette enrichie

Chaque nœud ou feuille de l'arbre contiennent une étiquette construite comme cela :

`etiq_formule(TypePrincipal, TypeSec1, TypeSec2, Formule, Polarité, Index)`

- **TypePrincipal** : Identifie la règle applicable (α, β pour la propositionnelle).
- **Types Secondaires** : Précisent le rôle des fils (ex : α_1 ou α_2).
- **Polarité** : Indique si la formule est affirmée (1) ou niée (0).
- **Index** : Un entier unique attribué selon un parcours en profondeur, permettant d'identifier précisément chaque position dans l'arbre.

2.1.2 Initialisation et Génération

Le prédicat `generer_arbre_indexe/2` lance le processus en initialisant la polarité à 0. Ce choix s'explique par la nature de la méthode des connexions : pour prouver qu'une formule est une tautologie, on cherche à montrer que sa négation (polarité 0) mène toujours à une contradiction.

```

1 % Initialisation de la construction
2 generer_arbre_index(Formule, Arbre) :-
3     regles_communes:etiqueter(Formule, 0, 0, _, none, none, Arbre).

```

Le prédicat fait appel au module `regles_communes` (décrit en Partie I) pour utiliser le prédicat d'étiquetage.

2.1.3 Récupération par les accesseurs

Pour manipuler cette structure complexe sans casser l'encapsulation, nous avons mis en place des accesseurs. Ils permettent d'extraire une information précise sans avoir à connaître l'ordre des arguments dans le terme `etiq_formule`.

```

1 % Exemples d'accesseurs
2 etiq_type_principal(etiq_formule(T, _, _, _, _, _), T).
3 etiq_formule(etiq_formule(_, _, _, F, _, _), F).
4 etiq_index(etiq_formule(_, _, _, _, _, I), I).

```

Nous avons mis en place ce système en prévoyant une potentielle évolution vers la logique des prédicats, sans avoir besoin de changer toute la logique métier.

2.2 Génération de l'arbre des chemins

Une fois l'arbre syntaxique indexé construit, l'étape suivante consiste à extraire l'ensemble des chemins sémantiques de la formule. Nous enregistrons ces chemins dans un arbre appelé **Arbre des chemins**.

2.2.1 Le développement de chemin

L'objectif est de parcourir l'arbre syntaxique et d'appliquer les règles de réduction de manière à isoler tous les ensembles de littéraux possibles qui, s'ils sont tous validés, prouvent la formule. Le cœur de cet algorithme réside dans le prédicat `developper/2`.

Ce prédicat manipule un *Ensemble* de nœuds issus de l'arbre indexé :

- **Cas de base** : Si l'ensemble ne contient plus que des feuilles (des littéraux), le chemin est complet. On crée une feuille `etiq_chemin_final` contenant la liste de ces littéraux.
- **Récursion** : On extrait le premier nœud de l'ensemble pour le traiter selon son type principal.

2.2.2 Traitement des règles α et β

La structure de l'arbre des chemins dépend directement de la nature du connecteur traité :

1. **Règle α (Conjonctive)** : Les composants de la formule α (ex : A et B pour $A \wedge B$) doivent appartenir au même chemin. On ajoute donc les fils à l'ensemble courant et on continue le développement linéairement.
2. **Règle β (Disjonctive)** : La formule β crée une bifurcation. Chaque fils de la règle β engendre un nouveau chemin indépendant.

2.2.3 Sémantique de l'Arbre des Chemins

Contrairement à l'arbre syntaxique qui représente la structure logique de la formule, l'arbre des chemins représente la **recherche de preuve** :

- Chaque nœud de cet arbre représente une étape de réduction (une application de règle).
- Chaque feuille de cet arbre représente un **chemin complet**.

Pour que la formule soit prouvée, chaque feuille de cet arbre des chemins devra contenir au moins une *connexion* (une paire de littéraux complémentaires). Cette structure permet une séparation claire entre la phase de décomposition logique et la phase de recherche de connexions que nous détaillerons dans la section suivante.

2.3 Recherche de connexions et validation

La dernière étape de la méthode des connexions en logique propositionnelle consiste à vérifier si l'arbre des chemins constitue une preuve valide. Pour qu'une formule soit prouvée, *tous* les chemins générés doivent être "fermés", c'est-à-dire qu'ils doivent contenir au moins une connexion.

2.3.1 Définition d'une connexion

Une connexion est une paire de littéraux (atomes) complémentaires présents sur un même chemin. Dans notre représentation, cela se traduit par deux feuilles ayant la même formule mais des polarités opposées (1 et 0).

Le prédicat `connexion/3` est chargé de trouver cette paire au sein d'une liste de feuilles. Il exploite le non-déterminisme de Prolog en utilisant `select/3` et `member/2` pour extraire deux éléments distincts, puis vérifie leurs propriétés via les accesseurs de l'étiquette :

```
1  % Recherche d'une paire complémentaire
2  connexion(Feuilles, F1, F2) :-
3      % Extraction d'une première feuille
4      select(F1, Feuilles, Reste),
5      feuille_etiquette(F1, Etiquette1),
6      etiq_formule(Etiquette1, Symbole),
7      etiq_polarite(Etiquette1, Polarite1),
8
9      % Calcul de la polarité opposée attendue
10     Polarite2 is 1 - Polarite1,
11
12     % Recherche d'une seconde feuille correspondante
13     member(F2, Reste),
14     feuille_etiquette(F2, Etiquette2),
15     etiq_formule(Etiquette2, Symbole),
16     etiq_polarite(Etiquette2, Polarite2).
```

2.3.2 Validation globale des chemins

Pour valider la preuve entière, l'algorithme doit parcourir l'arbre des chemins jusqu'à ses feuilles. Ce parcours est réalisé par le prédicat `tous_chemins_connectes/1`.

Lorsqu'il rencontre un nœud de branchement, il délègue récursivement la vérification à tous ses fils grâce à `maplist/2`. Lorsqu'il atteint une feuille finale (`etiq_chemin_final`), il vérifie qu'une connexion existe pour ce chemin :

```

1 % Verification sur un chemin final
2 tous_chemins_connectes(feuille(etiq_chemin_final(Feuilles))) :-
3     chemin_connecte(Feuilles).
4
5 % Recursion sur l'arbre des chemins
6 tous_chemins_connectes(noeud(_, ListeFils)) :-
7     maplist(tous_chemins_connectes, ListeFils).
```

Une optimisation importante réside dans le prédicat `chemin_connecte/1` (qui appelle `connexion/3`). L'utilisation d'un *cut* (!) permet de stopper la recherche dès qu'une première connexion est trouvée sur le chemin courant. En effet, une seule connexion suffit à fermer un chemin, il est donc inutile d'explorer les autres paires possibles.

2.3.3 Conclusion de la preuve

Le point d'entrée de ce processus est `verifier_connexions/2`. Il unifie son résultat avec l'atome `valide` si le prédicat `tous_chemins_connectes` réussit sur l'arbre complet, confirmant ainsi que la matrice est fermée (et donc que la formule de départ est bien une tautologie). Dans le cas contraire, si au moins un chemin reste ouvert, le résultat retourné est invalide.

3 En Logique du Premier Ordre (LPO)

La logique du premier ordre introduit la gestion des quantificateurs, ce qui complexifie significativement la méthode des connexions.

3.1 Adaptation de l'arbre indexé

3.2 Évolution de l'arbre des chemins

3.3 Recherche de connexions orientée LPO

3.4 Unification, graphe de dépendance et multiplicité

Conclusion